

Prof. Dr. Markus U. Mock

Studienprojekt *Entwicklung einer vollständigen Android (wahlweise iOS) App:*

Ausgabe:
TBD

Abgabe:
Hochladen des Source Codes und einer kurzen (ca. 500 Worte)
Dokumentation der App in Moodle bis Sonntag 18. Januar 2026
(Mitternacht MEZ)

Präsentation der Apps im Praktikum am 19. Januar 2026 (und falls zeitlich erforderlich Vorlesung am 21.01.26)

Inhaltsübersicht:

Überblick	2
Option 1: Fitness	3
Option 2: Geteilte Geldbörse	5
Option 3: Habit-Tracker	6
Option 4: Meine Finanzen	9
Option 5: Gamifizierter Tagesplaner	11

Überblick

Für das Studienprojekt entwickeln und dokumentieren Sie eine Android App **als Einzelarbeit**.

Diese wird nach folgenden Kriterien bewertet:

- Erfüllung der Anforderungen 30%
- Codequalität 15%: Einhalten der Kotlin/Swift Coding Conventions, Dokumentation z.B. Kommentare, angemessene Datenstrukturen)
- Software Design, insbesondere Einhaltung der Separation-of-Concerns 20%
- UI 20%: Fluidität der Oberfläche, Stabilität (keine Abstürze), minimal ansprechendes (sinnvolles) graphisches Design
- Dokumentation / Präsentation 15%: Vollständigkeit und Qualität der Beschreibung des Designs und von Besonderheiten der Umsetzung usw.

Im Folgenden werden die funktionalen Anforderungen der Apps genauer definiert. Sie können selbstverständlich über diese hinausgehen und auch in der Benutzerinterface sind Sie flexibel, soweit sie die beschriebenen Funktionalitäten zur Verfügung stellen.

Option 1: Fitness

Die Idee der Fitness App ist das Anlegen und Verfolgen von eigenen Fitness Übungen im Stile vieler verfügbarer Apps.

Funktionale Anforderungen: (Must)

1. Definition und Anlegen neuer Übungen mit Name und Bild. (CRUD)
2. Anzeige aller angelegten Übungen mit Name und Bild in einer Liste (Übungsansicht).
3. Eingabe von drei Übungssätzen mit einem Gewichts- und einem Wiederholungsparameter (Eingabeansicht).
4. Darstellung der eingegebenen Workoutparameter in einer Übersicht. (Workout Datenansicht)
5. Definition und Anlegen neuer Übungskategorien. (CRUD) mit Auflistung der Kategorien (Kategorieansicht).
6. Auflistung der dieser Kategorie zugeordneten Übungen nach einem Klick auf diese Kategorie.
7. Anzeige der Anzahl der gespeicherten Workouts, Übungen und Workout-Sets. (Workout-Set Ansicht)
8. Löschen aller Workouts einer Übung mit einem langen Klick.

Optionale Anforderungen

9. Graphische Ansicht des Übungsverlaufes
10. Reminder-Funktion zur Erinnerung an Übungen

Anbei ein paar Mockup-Bilder zur Illustration der Anforderungen.

<

+

x

Categories

Schulter

Rücken

Bauch

.

Infos zur Anzahl der gespeicherten Kategorien, Übungen, Workouts

Category

Enter Category

Add

<

+

x



Schulterheben



Seilzug über Kreuz



Item Three

<

Enter Name

Add



Add Workout Data

Aufwärmersatz

3

◀

▶

3

◀

▶

1. Satz

2. Satz

3. Satz

Add Workout

Eingefügte Daten

Name der Übung

Datum

1. Satz

2. Satz

3. Satz

Weitere Übungen hinzufügen

Workout abschließen

Kategorie 1:

Übung1 Datum

Übung2 Datum

Übung3 Datum

Kategorie2

Übung1 Datum

Übung2 Datum

D

Option 2: Geteilte Geldbörse

Die Idee der App ist das Erfassen und Verwalten von Ausgaben die man zusammen mit einem Partner macht. Wer kennt das Problem nicht, man hat mal nicht genügend Bargeld dabei und die Boiz nimmt nur cash. Später muss man sich daran erinnern und den anderen die Differenz ausbezahlen oder mit weiteren Ausgaben verrechnen. Die App soll solche Ausgaben für beide Partner erfassen und den Ausgleich erleichtern.

Funktionale Anforderungen: (Must)

1. Anlegung eines Nutzeraccounts mit Email, Name, (CRUD)
2. Einladen eines weiteren Nutzers mit dem man (bilateral) Ausgaben teilen kann, beliebig viele solcher Paarbeziehungen müssen möglich sein. Die Einladung ergeht an einen bereits existierenden Benutzer der eine Benachrichtigung von der App erhält. (Firebase Backend bietet sich dazu an).
3. Ausgaben mit Ort, Datum Uhrzeit an legen, und den vom Benutzer und vom Partner ausgegeben Betrag erfassen. Der Partner soll davon eine Benachrichtigung bekommen, und die Ausgabe als "akzeptiert" annehmen können. Ist eine Ausgaben nicht angenommen, soll dies visuell ersichtlich sein.
4. Alle Ausgaben eines Nutzers (alle Paare) bzw mit einem bestimmten Partner sollen angezeigt werden. Beschränkung auf bestimmte Zeiträume ist wünschenswert (optional).

Optionale Anforderungen

5. Neben der 50%-50% Aufteilung ist die Definition eines anderen Splits für Paare möglich, entweder mit Prozent oder mit Festbeträgen, z.B. A gibt 20€ aus, B 1€, es wird festgelegt, dass A für 15€ und B für 5€ verantwortlich ist, der Saldo wäre also dass B noch 4€ schuldet. Wichtig: bei mehreren Transaktionen beim selben Paar auf sollten für jede Ausgabe unterschiedliche Splits möglich sein, auf den Saldo achten.
6. Automatisierte Reminder an den Nutzer verschicken, der eine Ausgabe noch nicht akzeptiert hat (siehe Punkt 3).
7. Erweiterung auf Gruppenausgaben, d.h. eine Ausgabe wird nicht zwischen 2 Personen sondern N Personen aufgeteilt.

Option 3: Habit-Tracker

Die Idee der Habit-Tracker App ist das Anlegen, Verfolgen und Analysieren von persönlichen Gewohnheiten im Stil etablierter Apps wie Habitica oder Streaks, mit Fokus auf einfache Dokumentation, motivierende Visualisierungen und langfristige Fortschrittsverfolgung.

FUNKTIONALE ANFORDERUNGEN: (MUST)

1. Definition und Verwaltung neuer Gewohnheiten (vollständiges CRUD): Anlegen, Bearbeiten und Löschen von Gewohnheiten mit einem eindeutigen Namen; Zuordnung eines benutzerdefinierten Icons (z. B. durch Auswahl aus der Gerätegalerie oder Kameraaufnahme); Speicherung aller Gewohnheiten und zugehöriger Daten in Firebase (Realtime Database oder Firestore) für sichere Cloud-Synchronisation und Zugriff auf mehreren Geräten des Nutzers.
2. Anzeige aller angelegten Gewohnheiten in einer übersichtlichen Hauptansicht als interaktive Matrix: Jede Zeile repräsentiert eine Gewohnheit mit Icon, Name und aktuellem Streak-Status (z. B. laufende Serie an Erfolgen); Die Matrix ist flexibel skalierbar und scrollbar, um eine wachsende Anzahl von Gewohnheiten zu handhaben; Automatische Synchronisation mit Firebase für Echtzeit-Updates über Geräte hinweg.
3. Typisierung jeder Gewohnheit bei der Erstellung oder Bearbeitung: Wahl zwischen "Ja/Nein" (binäre Erfassung via Checkbox für einfache Abhaken-Aufgaben wie "Wasser trinken") oder "Zählbar" (numerische Eingabe für quantifizierbare Ziele wie "Schritte gehen" mit Zielwert).
4. Interaktive Tracking-Matrix in der Hauptansicht: Spalten repräsentieren Kalendertage (beginnend mit dem aktuellen Tag und erweiterbar auf vergangene und zukünftige Tage, z. B. die letzten 7–30 Tage); Jede Zelle am Schnittpunkt von Zeile (Gewohnheit) und Spalte (Tag) enthält ein anpassbares Eingabeelement für den täglichen Fortschritt, mit automatischer Speicherung und Validierung (z. B. Eingabebereich für Zählwerte) direkt in Firebase.
5. Direkte Fortschritterfassung in den Matrix-Zellen: Für "Ja/Nein"-Gewohnheiten eine intuitive Checkbox mit visueller Bestätigung (z. B. Häkchen-Animation); Für "Zählbare" Gewohnheiten ein Eingabefeld mit numerischer Tastatur, Zielvergleich (Erreicht/Nicht erreicht) und optionaler Freitext-Notiz für Kontext; Sofortige Firebase-Synchronisation für Multi-Device-Konsistenz.
6. Cloud-basierte Datenspeicherung und Synchronisation: Alle Gewohnheiten, Fortschrittseinträge, Streaks und Historien werden in Firebase persistent gespeichert; Unterstützung für Echtzeit-Sync, Offline-Caching (z. B. via Firebase SDK) und automatische Wiederherstellung bei Gerätewechsel; Integration von Firebase Authentication für benutzerspezifische Isolation der Daten.

7. Detailansicht pro Gewohnheit: Automatische Öffnung bei Auswahl einer Zeile in der Hauptansicht; Vollständige Historie als scrollbarer Kalender- oder Liste-View mit farblicher Markierung erfolgreicher Tage (z. B. Grün für Erfolge, Grau für Aussetzer), geladen aus Firebase; Anzeige von Metriken wie aktueller Streak, längster Streak, Gesamterfolgsrate und Durchschnitt pro Woche/Monat, mit Live-Updates.
8. Streak-Berechnung und -Visualisierung: Automatische Logik für positive (erfolgreiche aufeinanderfolgende Tage) und negative Serien (nicht erledigte Tage), berechnet und gespeichert in Firebase; Symbolische Darstellung in der Haupt- und Detailansicht: Feuersymbol für positive Streaks ab 3 Tagen, Aschesymbol für negative Serien ab 3 Tagen, Rauchsymbol als Vorläufer für 1–2 Tage (Symbole anpassbar); Integration in Streaks mit Benachrichtigungen bei Unterbrechungen.

OPTIONALE ANFORDERUNGEN

9. Erweiterte Visualisierungen in der Detailansicht: Graphische Darstellungen des Verlaufs, z. B. Balkendiagramm für wöchentliche Erfüllungsquoten, Liniendiagramm für Trendentwicklung oder Kreisdiagramm für monatliche Erfolge, mit Export-Option als Bild.
10. Dashboard-Übersicht mit Heatmap-Kalender: Separater Screen oder integrierte Sektion in der Hauptansicht; Kalender-View, in der jeder Tag farblich kodiert ist basierend auf der Anzahl erledigter Gewohnheiten (z. B. intensiveres Rot für hohe Produktivität); Filter- und Zoom-Funktionen für Monate/Jahre, mit Firebase-gestützter Aggregation.
11. Erinnerungsfunktion: Konfigurierbare tägliche Push-Benachrichtigungen pro Gewohnheit (z. B. Uhrzeit, Wiederholung, Textanpassung); Integration mit Geräteeinstellungen für Snooze oder Aussetzen, optional mit Firebase Cloud Messaging für Cross-Device-Versand.
12. Archivierungs- und Exportfunktion: Archivierung von abgeschlossenen Gewohnheiten (Ausblenden aus Hauptansicht, aber Zugriff über separates Archiv-Menü in Firebase); Beibehaltung aller historischen Daten; Optionale Export als CSV oder PDF für externe Analysen.
13. Sharing-Funktion: Teilen von Gewohnheiten oder Fortschrittsdaten mit anderen Nutzern via Firebase (z. B. Einladung per Link oder User-ID, Lesezugriff oder kollaborative Bearbeitung); Unterstützung für private Gruppen oder öffentliche Feeds.

Anbei ein Mockup-Bild zur Illustration der Anforderungen.

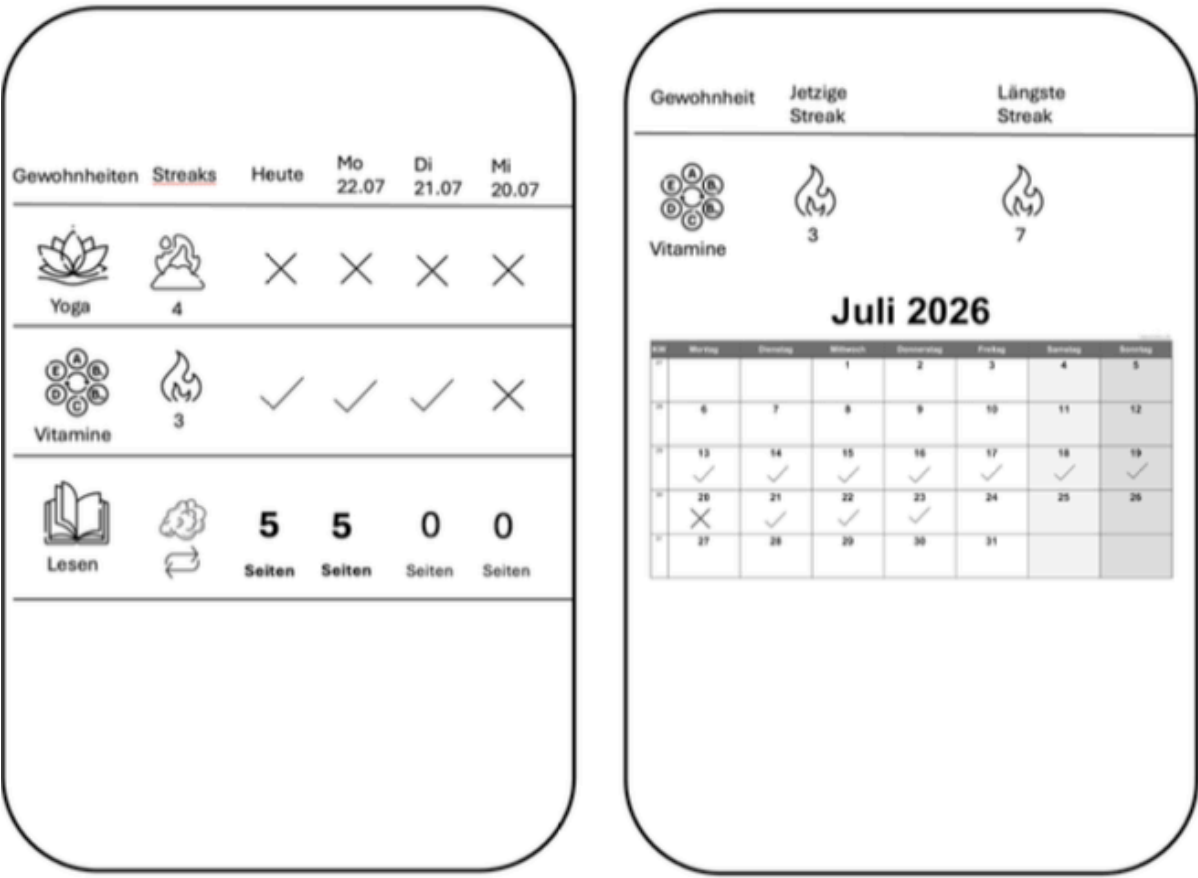


Figure 1: Mockup der Haupt- und Detailansicht des Habit-Trackers

Option 4: Meine Finanzen

Die Idee der Finanz-Tracker-App ist die manuelle Erfassung, Verwaltung und Analyse persönlicher Finanzen im Stil etablierter Apps wie Mint oder YNAB, mit Fokus auf strukturierte Übersichten, Kategorisierung und Warnfunktionen für Verträge und Budgets, für flexible, bankunabhängige Nutzung.

FUNKTIONALE ANFORDERUNGEN: (MUST)

1. Definition und Verwaltung von Finanz-Einträgen (vollständiges CRUD): Anlegen, Bearbeiten und Löschen von Einnahmen, Ausgaben, Verträgen und Schulden mit Details wie Betrag, Datum, Beschreibung und Gläubiger (bei Schulden); Speicherung aller Daten in Firebase (Realtime Database oder Firestore) für sichere Cloud-Synchronisation und Zugriff auf mehreren Geräten des Nutzers.
2. Kategorisierung von Einträgen: freie Erstellung und Zuordnung von Kategorien (z. B. Miete, Gehalt, Freizeit, Versicherung) bei der Eingabe; automatische Vorschläge basierend auf früheren Einträgen; Kategorien werden persistent in Firebase gespeichert und synchronisiert.
3. Listenansicht mit Filterung: Scrollbare Übersicht aller Einträge, filterbar nach Kategorie, Zeitraum (z. B. Monat, Jahr) oder Typ (Einnahmen/Ausgaben/Schulden/Verträge); Echtzeit-Lade und -Updates aus Firebase für konsistente Multi-Device-Anzeige.
4. Verträgeansicht und -Verwaltung: Dedizierte Übersicht mit allen gespeicherten Verträgen, inklusive Laufzeit, Betrag und Status; CRUD für Verträge mit automatischer Berechnung von Fristen; Daten in Firebase für Synchronisation und Offline-Caching via Firebase SDK.
5. Reminder-Funktion: Konfigurierbare Benachrichtigungen für Vertragsfristen (z. B. 30 Tage vor Ablauf), Budgetüberschreitungen pro Kategorie oder fällige Schuldentrückzahlungen; Integration mit Firebase Cloud Messaging für Cross-Device-Push-Benachrichtigungen und Snooze-Optionen.
6. Monats- und Jahresübersicht: Automatische Berechnung und Anzeige von Gesamtsummen für Einnahmen, Ausgaben, Saldo und Nettowachstum; Filterbare Views für Zeitperioden, mit Live-Aggregation aus Firebase-Daten.
7. Ausgabenanalyse: Darstellung der Ausgaben nach Kategorie und Zeitraum als sortierbare Tabelle oder Liste mit Prozentsätzen (z. B. "Freizeit: 25% des Monatsbudgets"); dynamische Berechnungen basierend auf synchronisierten Firebase-Daten.
8. Schuldenverwaltung: Erfassung von Schulden als dedizierte Kategorie mit Feldern für Betrag, Gläubiger, Rückzahlungsrate und Status (offen/teilweise/geschlossen); Tracking von Rückzahlungsfortschritt mit automatischer Aktualisierung in Firebase.

9. Exportansicht: Generierung strukturierter Listen oder Berichte aller Finanzdaten (z. B. als CSV oder PDF); direkte Vorschau und Download, mit Option zur Firebase-Synchronisation vor dem Export.

OPTIONALE ANFORDERUNGEN

10. Erweiterte Visualisierungen: graphische Analysen wie Kreisdiagramme für Ausgabenverteilung oder Liniendiagramme für Saldo-Entwicklung über Monate, mit Export als Bild; Aggregation aus Firebase für Echtzeit-Updates.
11. Dashboard-Übersicht: Start-Screen mit Heatmap-Kalender für tägliche/monatliche Finanzaktivitäten (z. B. Farbcodierung basierend auf Ausgabenintensität); Filter und Zoom für längere Perioden.
12. Budgetplanung: Erstellung monatlicher Budgets pro Kategorie mit Warnungen bei Überschreitungen; Speicherung und Synchronisation in Firebase.
13. Sharing-Funktion: Teilen von Finanzübersichten oder Schuldenlisten mit anderen Nutzern via Firebase (z. B. Einladung per Link oder User-ID, Lesezugriff für Beratung oder kollaborative Haushaltsverwaltung); Unterstützung für private Gruppen.
14. Mehrsprachigkeit: Vollständige Unterstützung für Deutsch und z.B. Spanisch, inklusive dynamischer UI-Übersetzungen, Währungsformate und Datumsanzeigen; Sprachauswahl im Einstellungsmenü, mit Firebase-gestützter Speicherung der Nutzerpräferenz.

Option 5: Gamifizierter Tagesplaner

Die Idee der gamifizierten Tagesplaner-App ist die motivierende Erfassung, Planung und Erledigung täglicher Tasks im Stil etablierter Apps wie Todoist oder Habitica, mit Fokus auf spielerische Elemente wie Punkte, Fortschritt und Belohnungen, um den Alltag fokussiert und unterhaltsam zu gestalten.

FUNKTIONALE ANFORDERUNGEN: (MUST)

1. Definition und Verwaltung von Tasks (vollständiges CRUD): Anlegen, Bearbeiten und Löschen von Tasks mit Pflichtfeldern wie Titel und Priorität (gering/mittel/hoch), sowie optionalen Feldern wie Beschreibung, Dauer und fester Startzeit; Speicherung aller Tasks und zugehöriger Daten in Firebase (Realtime Database oder Firestore) für sichere Cloud-Synchronisation und Zugriff auf mehreren Geräten des Nutzers.
2. Anzeige in einer Tagesübersicht als geordnete ToDo-Liste: Sortierung nach Priorität und fester Startzeit, mit Hervorhebung zeitlich kurz bevorstehender Tasks (z. B. Farbe oder Animation); Scrollbare, anpassbare Liste mit Echtzeit-Updates aus Firebase für Multi-Device-Konsistenz.
3. Statusanpassung pro Task: Intuitive Bedienung durch Antippen zur Umschaltung zwischen "Offen", "In Bearbeitung" und "Pausiert"; Checkbox für den Wechsel zu "Fertig" mit automatischer Aktualisierung in Firebase; Visuelle Indikatoren (z. B. Farbcodierung) für jeden Status.
4. Gamification-Elemente – Fortschrittsbalken: Dynamischer Balken, der über den Tag aktualisiert wird basierend auf erledigten Tasks (gewichtet nach Priorität oder Dauer); Live-Berechnung und Speicherung des Fortschritts in Firebase für persistente Darstellung.
5. Gamification-Elemente – Punkte-System: Vergabe von Punkten bei Fertigstellung (abhängig von Priorität oder Dauer); In-App-Shop für den Kauf virtueller Items (z. B. Bücherregal mit Büchern und Dekoration) unter Verwendung der Punkte; Punktestände und Inventar in Firebase synchronisiert.
6. Tagesabschluss-Bildschirm: Automatische Anzeige am Ende des Tages mit positiver Rückmeldung (z. B. motivierende Nachricht) und optionalen Statistiken (z. B. Erledigungsrate); Datenaggregation aus Firebase für personalisierte Inhalte.
7. Benachrichtigungen: Konfigurierbare Push-Erinnerungen zum Befüllen der ToDo-Liste (Nutzerwahl der Häufigkeit); automatische Alarmer für zeitgebundene Tasks (z. B. 5 Minuten vor Startzeit); Integration mit Firebase Cloud Messaging für Cross-Device-Versand und Snooze-Optionen.
8. Cloud-basierte Datenspeicherung und Synchronisation: Alle Tasks, Status, Punkte, Fortschritte und Statistiken werden in Firebase persistent gespeichert; Unterstützung für Echtzeit-Sync, Offline-Caching (via Firebase SDK) und

automatische Wiederherstellung bei Gerätewechsel; Integration von Firebase Authentication für benutzerspezifische Isolation der Daten.

OPTIONALE ANFORDERUNGEN

9. Erweiterte Visualisierungen: graphische Darstellungen wie Kreisdiagramme für Tageserledigungsquoten oder Liniendiagramme für Punkteentwicklung über Wochen, mit Export als Bild; Aggregation aus Firebase für Echtzeit-Updates.
10. Zeitplan-Funktion: Drag-and-Drop-Interface zum Einsetzen von Tasks als Bausteine in einen visuellen Tageszeitplan; automatische Kollisionswarnungen und Anpassungsvorschläge, synchronisiert in Firebase.
11. Pomodoro-Timer: Integrierter Timer für fokussierte Arbeitssessions (z. B. 25 Minuten Arbeit + 5 Minuten Pause), startbar allgemein oder task-spezifisch; Fortschrittsverfolgung in Firebase.
12. Badges und Achievements: Automatische Vergabe von Abzeichen für Meilensteine (z. B. bestimmte Anzahl erledigter Tasks, alle Tasks eines Tages fertig, 50% vor Mittag erledigt); Sammlung und Anzeige in einem Achievement-Menü, gespeichert in Firebase.
13. Streak- und Level-System: Tracking von aufeinanderfolgenden Erfolgen (Streaks) und Aufstieg durch Ränge (z. B. Schüler → Experte → Meister) basierend auf Punkten; visuelle Darstellung mit Benachrichtigungen bei Meilensteinen, in Firebase persistent.
14. Sharing-Funktion: Teilen von Task-Listen oder Erfolgsstatistiken mit anderen Nutzern via Firebase (z. B. Einladung per Link oder User-ID, Lesezugriff für Motivation oder kollaborative Planung); Unterstützung für private Gruppen.